One genre, two grammars: Community-conditioned subject omission in Git commit messages

English permits null subjects in restricted registers, most prominently diaries (Haegeman 1997, 2013; Weir 2012). Git commit subjects appear to add a technologically mediated case: *Fix parser* and *Add tests* are routinely called *imperatives* in developer style guides. That label is not innocent. An imperative analysis supplies both the missing subject (YOU) and the base verb, whereas a register-drop analysis treats omission as independently licensed by discourse and syntax. We show that the imperative diagnosis fails as a general analysis. The same communicative task is realized by two nearly categorical community conventions, one with a base verb and one with active past tense. The contrast extends register-conditioned subject omission to collaborative technical writing and shows that grammatical analysis of short digital texts must include community norms and interface metadata.

Data and method. We collected the latest 1,000 non-merge commit subjects from each of 14 major public repositories (14,000 subjects total) on November 5, 2026. The sample spans programming languages, web frameworks, and data-science projects. Bracketed review tags and one repository scope prefix (e.g., *net/http:*) were treated as metadata outside the clause. Automatic merges, backports, and explicit reverts were excluded. To avoid relying on a general parser trained on edited prose, we used a conservative, auditable criterion: the first clausal token had to instantiate one of 40 hand-specified English paradigms whose spelling distinguishes BASE, PAST, 3SG, and PROGRESSIVE (e.g., *fix/fixed/fixes/fixing*). This retained 8,291 subjects (59.2%). The criterion is transparent and exactly reproducible from the public commit histories.

Results. Of the retained subjects, 7,484 began with a base form, 752 with a past form, 23 with third-person singular, and 32 with a progressive. Table 1 reports the decisive base–past comparison. Thirteen repositories selected the base form in 97.4–100% of tokens; Django selected the past in all 685 base–past tokens. The association between repository and form is extremely large, $\chi^2(13) = 7442.9$, $p < .001$, Cramer's $V = .951$ ($N = 8{,}236$). Lexical choice cannot explain the split: *fix*, for example, occurs 2,213 times as *fix* and 537 times as *fixed* across the corpus. Nor are the past forms reduced passives: attested strings such as (1b) contain an overt direct object.

(1) a. *Fix host mismatch in the test.*
b. *Fixed #37388 – Improved unsafe user input examples.*

Table 1: Clause-initial base and past forms by repository.

Repository	Base	Past	Repository	Base	Past
CPython	564	1	Requests	643	16
React	496	3	NumPy	460	6
Django	0	685	pandas	281	2
Go	467	2	FastAPI	879	0
Node.js	455	0	Vue	887	2
PyTorch	433	3	Svelte	683	4
Flask	639	12	HTTPX	597	16

The Django distribution is independently corroborated by its public contributor guide, which explicitly requires past tense and illustrates *Fixed Unicode bug* (Django Software Foundation 2026); several other public guides prescribe the imperative base. These prescriptions are not noise imposed on grammar. They are evidence that a speech community conventionalizes the realization of a genre-specific construction.

Analysis. The past pattern cannot be an imperative: English imperatives lack past-tense morphology, and the transitive objects in (1b) exclude a passive parse. It instead instantiates written-register subject omission. The omitted argument is recoverable as the responsible developer or team, made continuously salient by the author field attached to every commit. Its clause is a root and the gap is leftmost once non-clausal interface labels are removed, matching core restrictions on diary drop (Haegeman 2013). Crucially, however, the base pattern is not thereby proven directive. A commit subject reports a completed change; it does not ordinarily direct its reader to perform one. We therefore analyze the two patterns as competing allostructions of a COMMIT-REPORT construction:

$$\left[\textsc{commit-report}\ \varnothing_{\text{author/team}}\ V_{\{\text{base,past}\}}\ XP\right].$$

The genre licenses a recoverable left-edge subject, while repository-level convention selects the verbal realization. On this account, *imperative* is a useful style label but an unreliable morphosyntactic diagnosis: base morphology, null subject, and directive force come apart.

This analysis makes three testable predictions. First, authors moving between repositories should accommodate their verb form to the destination community. Second, removing visible authorship metadata should reduce the acceptability of past-form commit reports more than overt-subject paraphrases. Third, material that is syntactically clause-internal, unlike scope labels, should block left-edge omission in the past construction. More broadly, the result supports a view of digital registers in which platform fields participate in recoverability, but community practice determines which grammatical resources realize the same discourse function.

References

[Django Software Foundation 2026] Django Software Foundation. 2026. Committing code. `https://docs.djangoproject.com/en/dev/internals/contributing/committing-code/`.

[Haegeman 1997] Haegeman, Liliane. 1997. Register variation, truncation, and subject omission in English and French. *English Language and Linguistics* 1(2). 233–270.

[Haegeman 2013] Haegeman, Liliane. 2013. The syntax of registers: Diary subject omission and the privilege of the root. *Lingua* 130. 88–110.

[Weir 2012] Weir, Andrew. 2012. Left-edge deletion in English and subject omission in diaries. *English Language and Linguistics* 16. 105–129.